

Certaines questions comptent pour l'UE2.1 (17.5 points) alors que d'autres comptent pour l'UE2.2 (8.5 points). Elles sont clairement identifiées dans le sujet et donneront lieu à deux notes distinctes.

Par ailleurs, une annexe est disponible à la fin du sujet et contient la description des méthodes dont vous pourriez avoir besoin.

Inutile de fournir les instructions d'import. Sauf explicitement demandé, vous pouvez supposer que tous les accesseurs dont vous avez besoin sont disponibles : pas besoin de les écrire.

Vous êtes libre de décomposer votre code et d'ajouter des méthodes dans ce but. À vous de penser à la surcharge ou au chaînage ! Par ailleurs, si pour répondre à une question, vous avez besoin d'écrire du code dans différentes classes, n'oubliez pas de le spécifier clairement avant votre code.

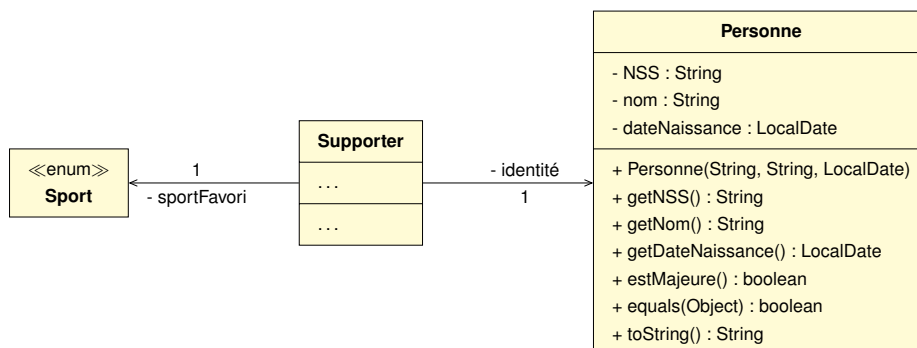
## Exercice 1 : Épreuves lilloises pour les jeux olympiques

Les jeux olympiques ont lieu cette année en France et certaines épreuves ont lieu ici même, à Lille ! On s'intéresse ici à la vente de tickets aux supporters, uniquement pour les épreuves qui ont lieu localement. Elles sont trois : HANDBALL, BASKET et la célèbre épreuve olympique de KARAOKÉ.

**Q1.** Écrivez l'énumération `Sport` qui regroupe ces épreuves.

Les supporters sont caractérisés par un nom, une date de naissance et un sport favori. Heureux fruit du hasard, vous disposez d'une classe `Personne` parfaitement fonctionnelle qui exhibe un certain nombre de points communs. En effet, une personne est caractérisée par un numéro de sécurité sociale, un nom et une date de naissance. De manière avisé, vous décidez d'encapsuler la classe `Personne` même si le numéro de sécurité sociale ne sera ni renseigné ni utilisé chez un supporter. Cette classe `Personne` dispose déjà de tous ses accesseurs, d'une méthode booléenne `estMajeure`, d'une méthode `equals` ainsi que d'une méthode `toString` retournant une personne sous la forme : "[<nss>] <nom> (<dateNaissance>)".

**Q2.** Écrivez la classe `Supporter` en respectant les spécifications UML suivantes :



**Q3.** Munissez cette classe `Supporter` d'un constructeur complètement paramétré.

**Q4.** Ajoutez les accesseurs pour les attributs `dateNaissance` et `sportFavori`. On supposera dans la suite que tous les autres accesseurs sont disponibles même si vous ne les avez pas écrit.

**Q5.** Ajoutez à votre classe `Supporter` une méthode `estMajeur` qui détermine si un supporter est majeur ou ne l'est pas.

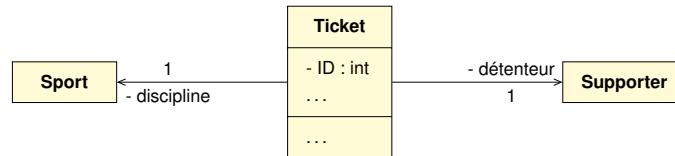
```
boolean estMajeur()
```

**Q6.** Ajoutez une méthode `equals` à la classe `Supporter` qui se base sur tous ses attributs.

```
boolean equals(Object obj)
```

**Q7.** Écrivez une méthode `toString` retournant un supporter sous la forme : "`[<sportFavori>] <nom> (<dateNaissance>)`". Par exemple, "`[BASKET] Alice (2042-04-02)`" correspond à une supportrice prénommée *Alice*, fan de basket, née le *2042-04-02*.

Maintenant, attelons-nous à la création d'un ticket. Les tickets sont identifiés par un numéro unique (quel que soit le sport concerné), généré via un numéro automatique. Il est nominatif et donc attribué à un supporter spécifique. Chaque ticket est "*open*" et porte sur une discipline complète : il ne comporte pas de date ni de place. Il est utilisable pour n'importe quel évènement de la discipline en question durant les Jeux Olympiques. Voici sa représentation UML :



**Q8.** Écrivez la classe `Ticket` répondant à ces spécifications UML. Ajoutez à cette classe `Ticket` un constructeur complètement paramétré.

```
Ticket(Supporter détenteur, Sport discipline)
```

**Q9.** Ajoutez une méthode **statique** à la classe `Ticket`, nommée `mêmeDétenteur`, qui permet de déterminer si deux tickets ont le même détenteur. Autrement dit, cette méthode booléenne détermine si un même supporter possède deux tickets. À vous de déterminer sa signature !

On peut finalement s'intéresser à la plateforme de réservation des billets. Même si les tickets sont tous numérotés de manière unique, on souhaite stocker les billets par sport. Pour cela, on utilisera un **tableau** nommé `réservations` dont chaque case contient une **liste** des tickets (`ArrayList<Ticket>`) liées à un sport donné. Pour simplifier, on suppose que les listes dédiées chacune à un sport sont dans le même ordre que l'énumération `Sport`. Il y a donc autant de listes qu'il y a de sports, comme l'illustre l'exemple décrit Figure 1.

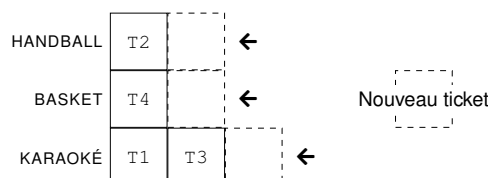


FIGURE 1 – Structure interne de la plateforme où l'on voit un ticket `T2` réservé pour le handball, un ticket `T4` pour le basketball, et deux tickets `T1` et `T3` pour l'épreuve de karaoké. Chaque nouveau ticket est ajouté à la liste des tickets du sport correspondant.

**Q10.** Écrivez la classe `Plateforme` munie d'un constructeur sans paramètre qui initialise simplement son attribut. Notez que les listes sont vides à la création d'une instance, et que le nombre de sport à prendre en compte est défini dans l'énumération `Sport`.



**Q11.** Ajoutez les méthodes `nbRéservations` nécessaires pour savoir combien de réservations ont été réalisées, soit selon un sport donné, soit tout sport confondu.

```
int nbRéservations(Sport unSport)
int nbRéservations()
```

**Q12.** Les supporters peuvent acheter leurs tickets de différentes manières. Ils peuvent l'acheter soit auprès d'un revendeur partenaire agréé qui émet le ticket, il ne nous reste plus qu'à l'enregistrer : le ticket est alors passé en paramètre.

L'autre alternative est de l'acheter directement sur la plateforme : il faut alors créer le ticket à partir des informations fournies. Écrivez deux méthodes d'ajout `ajoutRéservation` qui ajoute ou crée le ticket à votre plateforme. On notera qu'un supporter ne va pas nécessairement voir les événements liés à son sport favori et peut en conséquence acheter des tickets liés à n'importe quelle épreuve.

```
void ajoutRéservation(Ticket unTicket)
void ajoutRéservation(Supporter unSupporter, Sport unSport)
```

**Q13.** Certaines épreuves peuvent s'avérer violentes et choquer un public plus sensible. On souhaite donc limiter la création d'un ticket à des supporters majeurs uniquement dans ces cas là. Écrivez une méthode d'ajout plus sécurisée, nommée `ajoutSécuriséRéservation`, qui vérifie que le supporter est majeur si le sport demandé est le karaoké. Aucun ticket n'est instancié pour un mineur dans ce cas et la méthode retourne `false`. Face aux autres sports, le ticket peut être créé sans crainte, le ticket ajouté et la méthode retourne `true`.

```
boolean ajoutSécuriséRéservation(Supporter unSupporter, Sport unSport)
```

**Q14.** On souhaite pouvoir récupérer un ticket à partir de son identifiant. Écrivez la méthode `getTicket` qui retourne un ticket à partir de son numéro d'identifiant.

```
Ticket getTicket(int id)
```

**Q15.** Le gouvernement a procédé à des tirages au sort pour la vente de billets. Il souhaite désormais savoir combien de supporters, fan d'un sport, ont pu assister à une épreuve de ce même sport. Veillez à décomposer votre code comme il convient en précisant explicitement les classes concernées.

```
int nbFanSatisfait()
```



## Annexe : Méthodes utiles

### Classe ArrayList<E>

|         |                                                                                                                                                                                                            |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| boolean | <code>add(E e)</code><br>Appends the specified element to the end of this list.                                                                                                                            |
| void    | <code>add(int index, E element)</code><br>Inserts the specified element at the specified position in this list.                                                                                            |
| boolean | <code>addAll(Collection&lt;E&gt; c)</code><br>Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's Iterator. |
| boolean | <code>contains(Object o)</code><br>Returns true if this list contains the specified element.                                                                                                               |
| E       | <code>get(int index)</code><br>Returns the element at the specified position in this list.                                                                                                                 |
| int     | <code>indexOf(Object o)</code><br>Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.                                        |
| boolean | <code>isEmpty()</code><br>Returns true if this list contains no elements.                                                                                                                                  |
| int     | <code>lastIndexOf(Object o)</code><br>Returns the index of the last occurrence of the specified element in this list, or -1 if this list does not contain the element.                                     |
| E       | <code>remove(int index)</code><br>Removes the element at the specified position in this list.                                                                                                              |
| boolean | <code>remove(Object o)</code><br>Removes the first occurrence of the specified element from this list, if it is present.                                                                                   |
| boolean | <code>removeAll(Collection&lt;E&gt; c)</code><br>Removes from this list all of its elements that are contained in the specified collection.                                                                |
| E       | <code>set(int index, E element)</code><br>Replaces the element at the specified position in this list with the specified element.                                                                          |
| int     | <code>size()</code><br>Returns the number of elements in this list.                                                                                                                                        |

### Classe Arrays

|                                   |                                                                                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>static List&lt;T&gt;</code> | <code>asList(T... a)</code><br>Returns a fixed-size list backed by the specified array.                                                                               |
| <code>static T[]</code>           | <code>copyOf(T[] original, int newLength)</code><br>Copies the specified array, truncating or padding with nulls (if necessary) so the copy has the specified length. |
| <code>static T[]</code>           | <code>copyOfRange(T[] original, int from, int to)</code><br>Copies the specified range of the specified array into a new array.                                       |
| <code>static void</code>          | <code>sort(Object[] a)</code><br>Sorts the specified array of objects into ascending order, according to the natural ordering of its elements.                        |
| <code>static String</code>        | <code>toString(Object[] a)</code><br>Returns a string representation of the contents of the specified array.                                                          |

### Classe String

|         |                                                                                                                                  |
|---------|----------------------------------------------------------------------------------------------------------------------------------|
| boolean | <code>contains(CharSequence c)</code><br>Returns true if and only if this string contains the specified sequence of char values. |
| boolean | <code>equalsIgnoreCase(String anotherString)</code><br>Compares this String to another String, ignoring case considerations.     |
| String  | <code>strip()</code><br>Returns a string whose value is this string, with all leading and trailing white space removed.          |
| String  | <code>substring(int beginIndex)</code><br>Returns a string that is a substring of this string.                                   |
| String  | <code>substring(int beginIndex, int endIndex)</code><br>Returns a string that is a substring of this string.                     |
| String  | <code>toLowerCase()</code><br>Converts all of the characters in this String to lower case.                                       |
| String  | <code>toUpperCase()</code><br>Converts all of the characters in this String to upper case.                                       |

### enum E

|            |                                                                                                                                                                               |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| int        | <code>compareTo(E o)</code><br>Compares this enum with the specified object for order.                                                                                        |
| boolean    | <code>equals(Object other)</code><br>Returns true if the specified object is equal to this enum constant.                                                                     |
| String     | <code>name()</code><br>Returns the name of this enum constant, exactly as declared in its enum declaration.                                                                   |
| int        | <code>ordinal()</code><br>Returns the ordinal of this enumeration constant (its position in its enum declaration, where the initial constant is assigned an ordinal of zero). |
| String     | <code>toString()</code><br>Returns the name of this enum constant, as contained in the declaration.                                                                           |
| static E   | <code>valueOf(String name)</code><br>Returns the enum constant of the specified name.                                                                                         |
| static E[] | <code>values()</code><br>Returns an array containing all possible values defined by the enum.                                                                                 |

### Classe LocalDate

|                  |                                                                                                                                                                                         |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| boolean          | <code>isAfter(ChronoLocalDate other)</code><br>Checks if this date is after the specified date.                                                                                         |
| boolean          | <code>isBefore(ChronoLocalDate other)</code><br>Checks if this date is before the specified date.                                                                                       |
| static LocalDate | <code>now()</code><br>Obtains the current date from the system clock in the default time-zone.                                                                                          |
| static LocalDate | <code>of(int year, int month, int dayOfMonth)</code><br>Obtains an instance of LocalDate from a year, month and day.                                                                    |
| LocalDate        | <code>plusDays(long daysToAdd)</code><br>Returns a copy of this LocalDate with the specified number of days added.                                                                      |
| long             | <code>until(LocalDate endExclusive, TemporalUnit unit)</code><br>Calculates the amount of time until another date in terms of the specified unit (e.g., <code>ChronoUnit.YEARS</code> ) |